

Load Testing Report

Tool: Apache JMeter 5.6.3

API: POST /booking

Test Type: Load / Performance Testing

1. Objective

The purpose of this test was to check whether the booking API can handle around 5000 requests per minute (about 83 requests per second), while keeping the average response time below 2 seconds. I also wanted to identify any performance issues and record a baseline for future testing.

2. Test Setup

- 50 concurrent users were used.
- Ramp-up period was 10 seconds.
- The test duration was 60 seconds.
- A Constant Throughput Timer was configured for 5,000 samples per minute.
- The request used Content-Type: application/json and Accept: application/json.

3. Test Results

Metric	Result
Total Requests	4768
Average Response Time	285 ms
Minimum Response Time	232 ms
Maximum Response Time	1271 ms
Standard Deviation	90.44 ms
Error Rate	0.00%
Achieved Throughput	79.2 requests/sec
Approx. Requests/Minute	4752

The API handled the test load without any failed requests. The average response time was 285 ms, which is well below the required 2-second limit. The test achieved 79.2 requests per second, which is close to the target of about 83 requests per second.

4. Bottleneck Analysis

No clear breaking point was found during this test. The error rate stayed at 0% and the response time remained low. However the achieved throughput was slightly below the 5000 requests-per-minute target. This may be affected by the local test machine, network connection, public API environment or server capacity.

Possible Bottleneck 1 - Server Processing

Symptoms: Response times increase and throughput starts to drop.

Possible cause: The application server may have high CPU usage or slow processing.

Recommendation: Monitor CPU usage, optimize application processing, and scale the server if required.

Possible Bottleneck 2 - Database

Symptoms: Requests become slower when many users access the API at the same time.

Possible cause: Slow database queries, locking or limited database connections may affect performance.

Recommendation: Optimize queries, review indexes and tune the database connection pool.

Possible Bottleneck 3 - Network / Infrastructure

Symptoms: Response times become inconsistent or the expected request rate cannot be reached.

Possible cause: Network latency, bandwidth limitations, rate limits or infrastructure constraints may exist.

Recommendation: Monitor network performance, check rate limits and use a dedicated load generator for larger tests.

5. Conclusion

Overall the API performed well during the test. It processed 4768 requests in the 60-second run with an average response time of 285 ms and a 0% error rate. The achieved throughput was 79.2 requests per second, which is slightly below the target of about 83 requests per second. Based on this run, the API was stable at the tested load, but another test using a dedicated load-testing environment would be useful to confirm the full 5000 requests-per-minute target.

6. Evidence to Include in Submission

- JMeter .jmx test plan file.
- Screenshot of the final Summary Report.
- This load testing report.